



SOP/CORS

Lecture 25
Mahhek Tahir



Imagine you are logged into your site in one tab...

Now you open another website.

Should that website be allowed to read your bank data?

That's where **Same-Origin Policy (SOP)** comes in.
A website can only access data from its own origin.



Same-Origin Policy (SOP)

Same-Origin Policy is a **browser security rule** that says:

A webpage can only read data from the same origin it was loaded from.

SOP is enforced by the browser, not the server.



What an origin?

Origin = **protocol + domain + port + subdomain**

URL 1	URL 2	Same Origin?	Why
http://example.com	http://example.com	✓ Yes	protocol, host, port same
http://example.com	https://example.com	✗ No	protocol different
https://example.com	https:// app.example.com	✗ No	subdomain (host) different
http://example.com:3000	http:// example.com:5000	✗ No	port different
https:// app.example.com:443	https:// app.example.com	✓ Yes	same (443 is default HTTPS port)

SOP



A webpage is loaded from an origin (protocol + domain + port), e.g.
`http://localhost:3000`

The webpage makes a request to another origin, e.g.
`http://localhost:5000`

The browser attaches the origin of the request:

Origin: `http://localhost:3000`

The request is sent to the server successfully (SOP does not block sending requests)

The server processes the request and returns a response

Example:

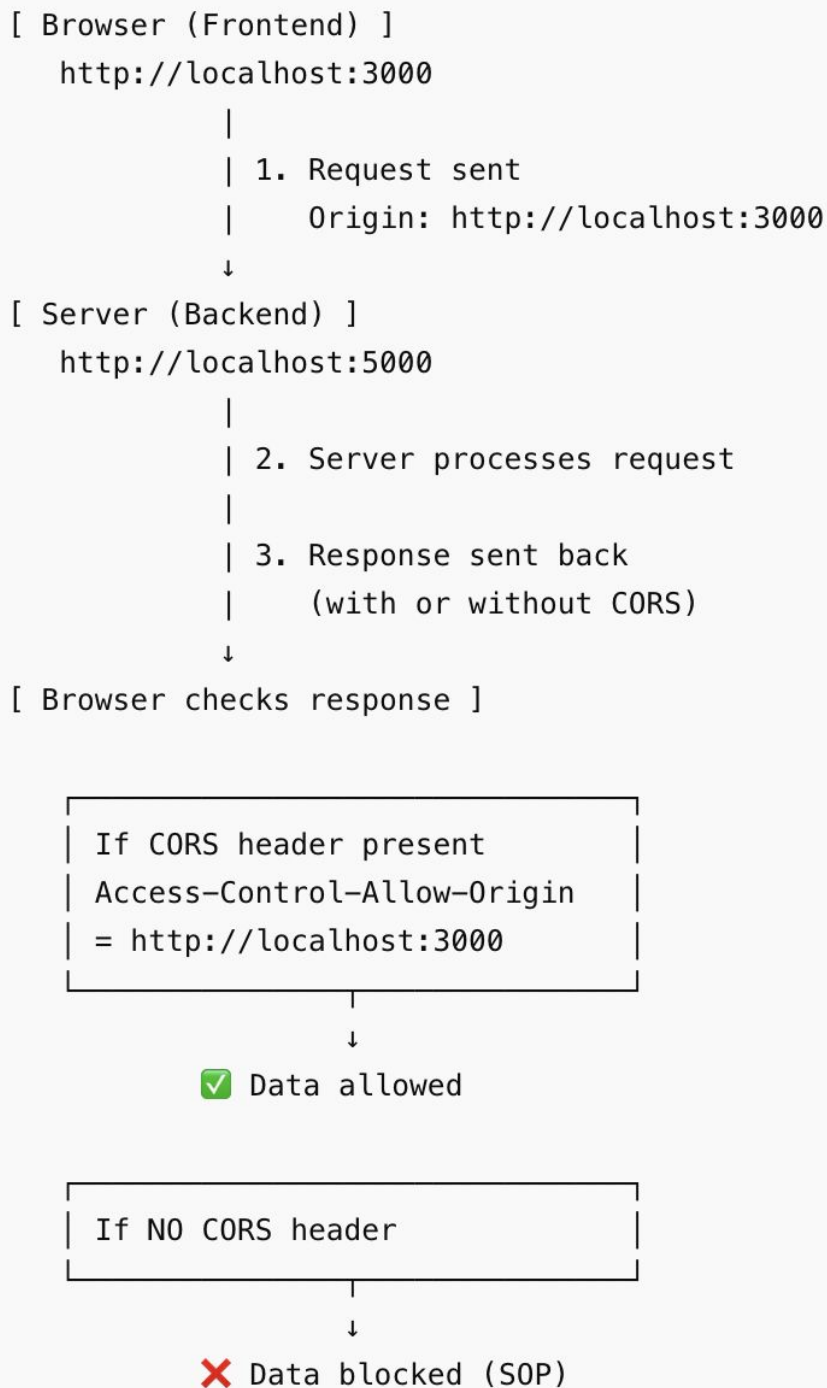
Your frontend (running on 3000): `fetch("http://localhost:5000/data")`

This means: Page from **3000** is asking data from **5000**



What SOP Blocks

Browser always sends request, but only allows response if server permits (CORS).



SOP



The browser receives the response and checks:

- Does the response include CORS headers?
- Does the allowed origin match the requesting origin?

If no valid CORS headers are present:

- The browser blocks access to the response in JavaScript

If valid CORS headers are present:

- The browser allows JavaScript to access the response

Important:

- SOP does not stop requests
- SOP only controls whether the response can be read by the browser
- Access to fetch at '...' from origin '...' has been blocked by CORS policy



The Problem (Real Scenario)

You built a React frontend and a Node backend...

- React → `localhost:3000`
- Node → `localhost:5000`

Access to fetch has been blocked by CORS policy

but both are my sites?

Browser doesn't trust your intention — it enforces rules strictly.



Why does browser do this?

To stop attacks like:

- *Malicious site reading your data*
- *Accessing APIs using your login*

Usecase



Your website: <https://myapp.com>

You try to get data from: <https://api.food.com/pizza>

```
fetch("https://api.food.com/pizza")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

If CORS is NOT allowed

- Request goes to `api.food.com`
- Server sends pizza data back

BUT browser says: Blocked by CORS policy

- `console.log(data)` never runs, No data shown on screen, You can't use the response

If CORS IS allowed

Server sends header: Access-Control-Allow-Origin: <https://myapp.com>

- Data received, `console.log(data)` works, You can show pizza on UI



CORS Decision Cases

Case 1 — Same Origin (Allowed)

Request:

`http://localhost:5000 → http://localhost:5000`

- Same protocol, domain, port
- No CORS needed
- Browser allows response



Case 2 — Different Port (Blocked without CORS)

Request:

`http://localhost:3000 → http://localhost:5000`

- Different port = different origin
- No CORS header

Result:

- Request is sent
- Response is blocked by browser



Case 3 — Different Port with CORS (Allowed)

Request:

Origin: http://localhost:3000

Response:

Access-Control-Allow-Origin: http://localhost:3000

Result:

- Origin matches
- Browser allows response

“Server allows the client’s **origin** to read the response.



Case 4 — Different Site / Mismatch (Blocked)

Request:

Origin: `http://evil.com`

Response:

Access-Control-Allow-Origin: `http://localhost:3000`

Result:

- Origin does not match
- Browser blocks response

Key Rule

- Same origin → always allowed
- Different origin → requires correct CORS header
- Missing or mismatched header → blocked



Any origin (open to all)

Access-Control-Allow-Origin: *

not safe

allows all



CORS protects users, not your backend

It prevents malicious websites from reading data in the browser

It does NOT stop requests from tools like Postman or curl
Backend APIs must still implement authentication and validation



CORS

Without CORS

```
app.get("/data", (req, res) => {  
  res.json({ message: "Hello" });  
});
```

- Request from another origin is sent successfully
- Server responds normally
- No CORS headers in response

Result:

- Permissions are **declared by the server and enforced by the browser.**



CORS

With CORS Enabled

```
const cors = require("cors");  
app.use(cors());
```

Server adds header:

Access-Control-Allow-Origin: * //I (server) allow **any origin** to read my response in the browser

Meaning:

- Allow all origins to access this resource

Result:

- Browser allows JavaScript to read the response

Restricting CORs



Restricting CORS (Best Practice)

```
app.use(cors({  
  origin: "http://localhost:3000"  
})); //I (server) allow this origin to read my response in the browser.
```

Meaning:

- Only this origin is allowed

Result:

- Allowed origin → access granted
- Other origins → response blocked by browser

Important Understanding

- Server always sends the response
- Browser decides whether it can be accessed
- CORS is a permission system between server and browser



```
const express = require("express");
const cors = require("cors");

const app = express();

const allowedOrigins = [
  "http://localhost:3000",
  "http://localhost:5173",
  "https://myfrontend.com"
];

app.use(cors({
  origin: function (origin, callback) {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true); // allow
    } else {
      callback(new Error("Not allowed by CORS"));
    }
  }
}));
```



SOP VS CORS

Key Difference

Feature	SOP	CORS
Type	Restriction	Permission mechanism
Controlled by	Browser	Server (via headers)
Default behavior	Block cross-origin	Allow only if configured
Role	Security enforcement	Controlled access

Same-Origin Policy (SOP) vs CORS



Same-Origin Policy (SOP)

- Browser security rule
- Blocks JavaScript from reading responses from different origins
- Server has NO control over SOP
- It is automatically applied
- Default behavior: **deny cross-origin access**

Purpose:

- Protect users from malicious websites accessing sensitive data

CORS (Cross-Origin Resource Sharing)

- Mechanism to relax SOP restrictions
- Implemented by the server using HTTP headers
- Allows specific origins to access resources

Purpose:

- Enable controlled cross-origin communication